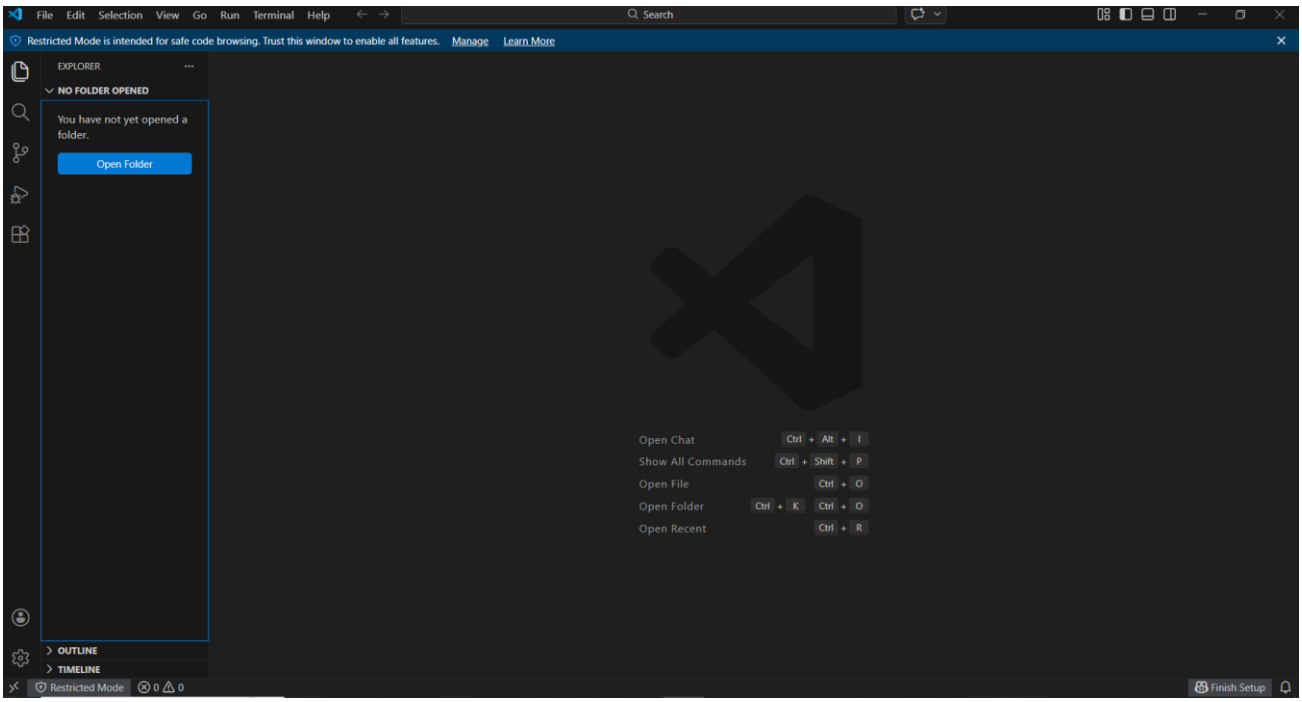
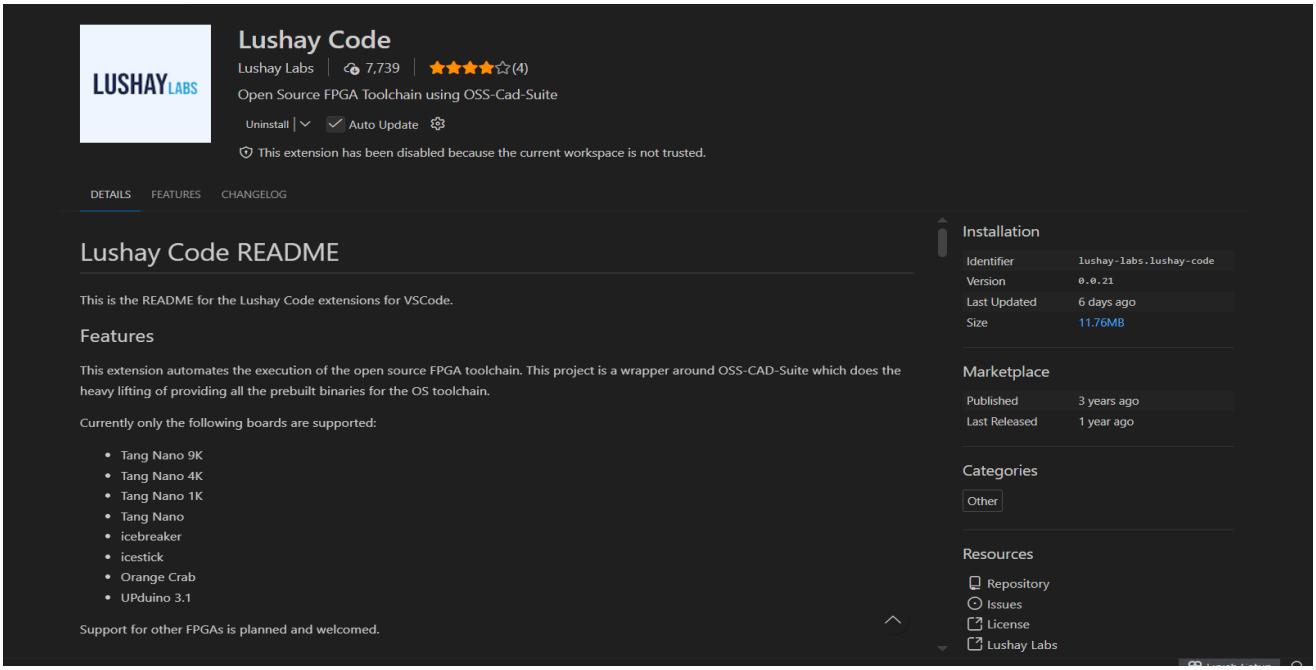
Jose Daniel Marchena Faerron Tutorial

Sección 1: Instalación de herramientas

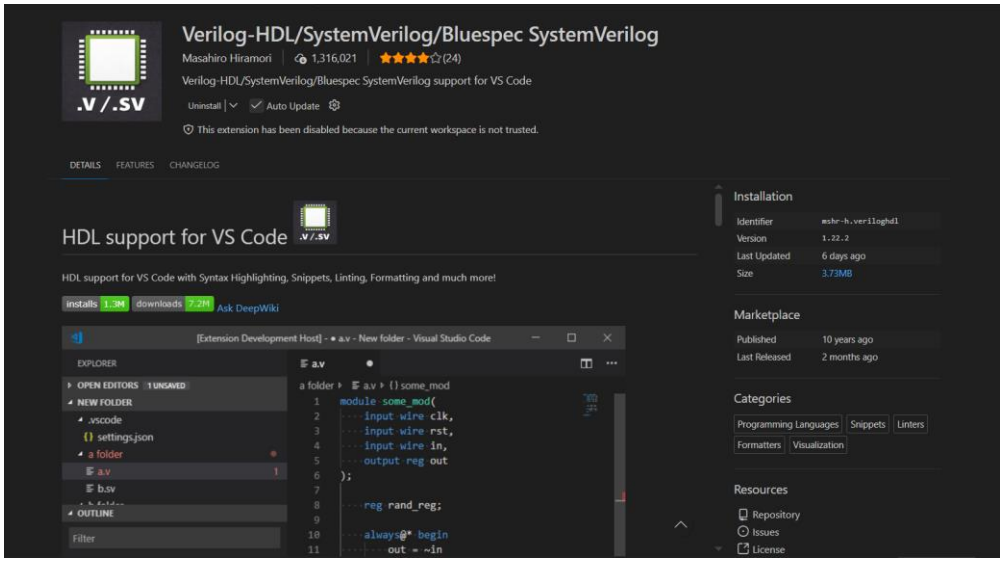
Instalación del Visual Studio Code:



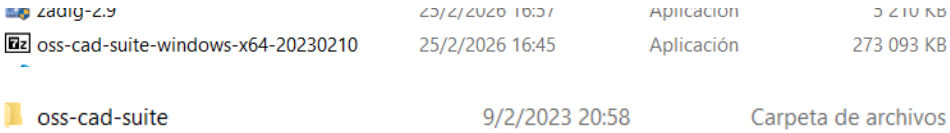
Instalación del *Lushay Code*:



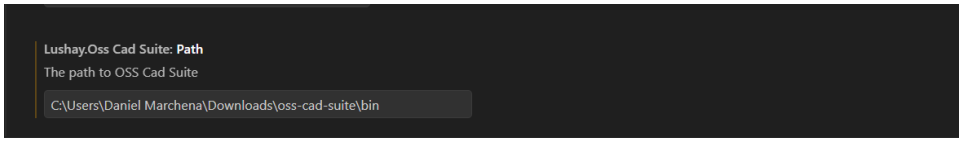
Instalación del *Verilog-HDL/SystemVerilog :*



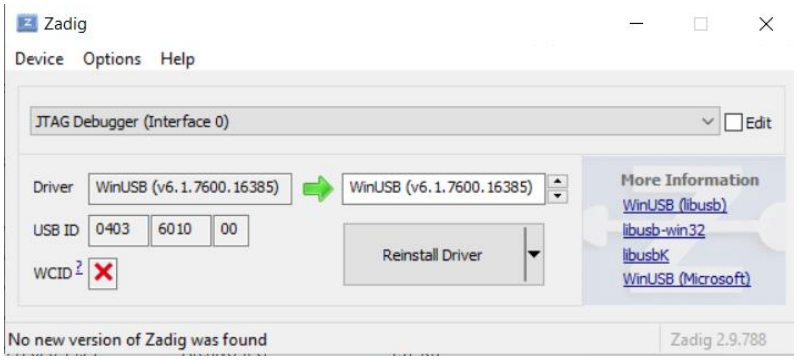
Instalación del OSS-Cad-Suite de YosysHQ:



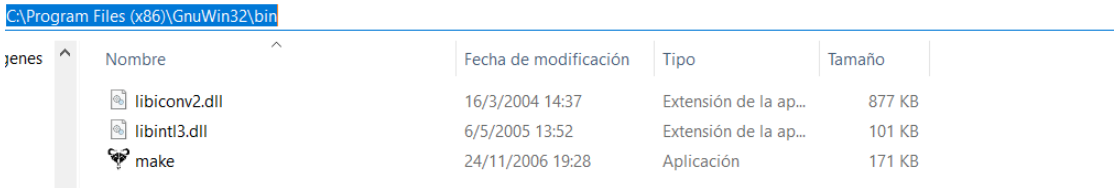
Configuración de la extensión Lushay Code:

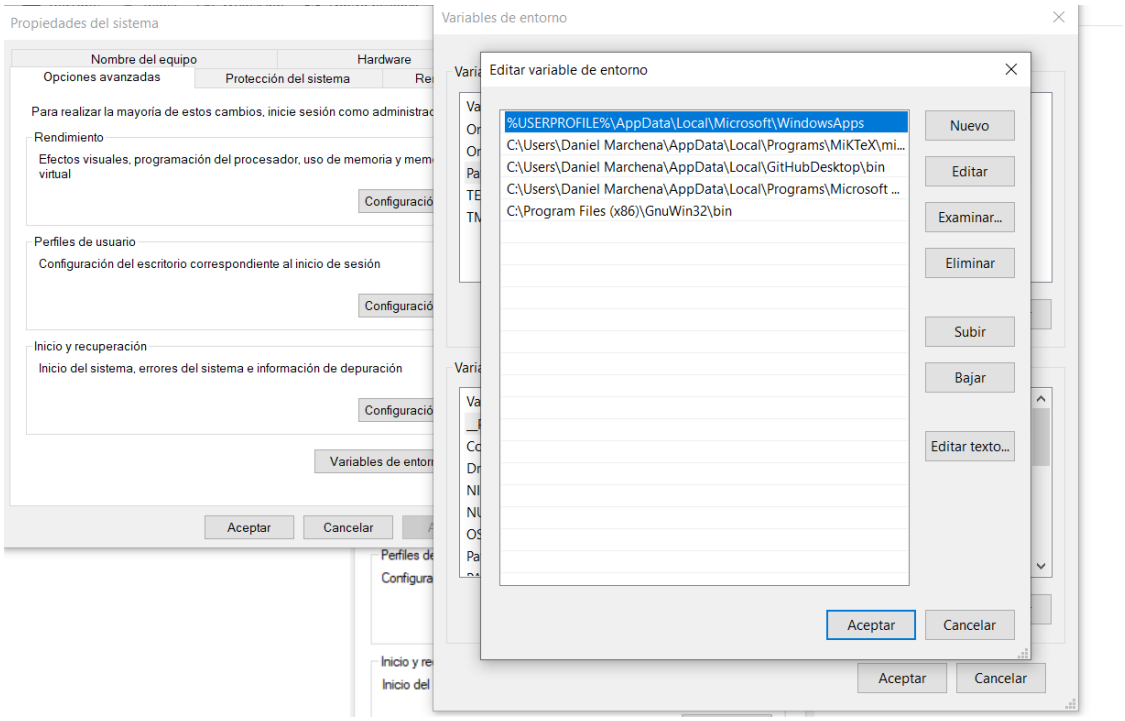


Configuración del Driver USB de Windows



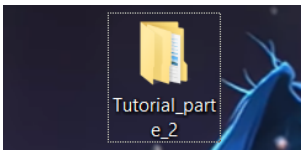
Instalación de GNU-Make:



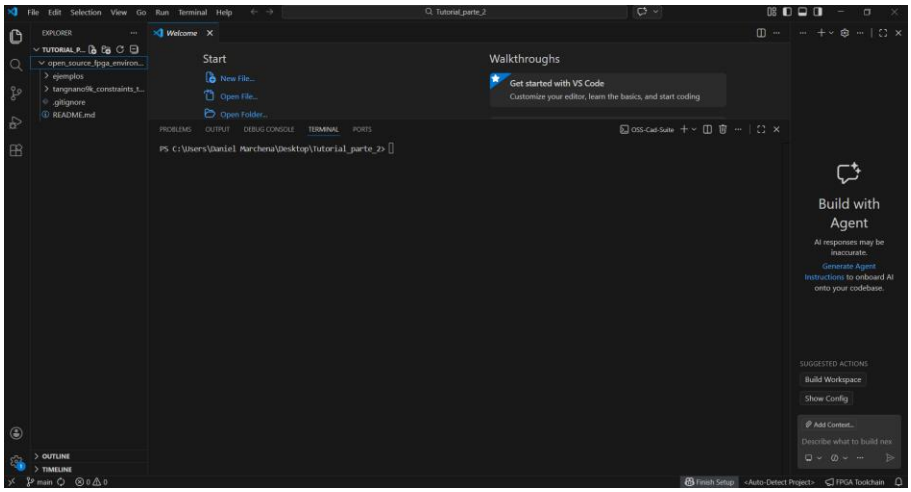


Sección 2. Uso del toolchain para diseño en FPGA

Clonación y ubicación del repositorio:



Inicializar la terminal del toolchain:



Comando Yosys y salida de Yosys:

```
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2> yosys

 /----------------------------------------------------------------------------\
 |                                                                            |
 |  yosys -- Yosys Open SYnthesis Suite                                       |
 |                                                                            |
 |  Copyright (C) 2012 - 2020  Claire Xenia Wolf <claire@yosyshq.com>         |
 |                                                                            |
 |  Permission to use, copy, modify, and/or distribute this software for any  |
 |  purpose with or without fee is hereby granted, provided that the above    |
 |  copyright notice and this permission notice appear in all copies.         |
 |                                                                            |
 |  THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES  |
 |  WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF          |
 |  MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR   |
 |  ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES    |
 |  WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN     |
 |  ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF   |
 |  OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.            |
 |                                                                            |
 \----------------------------------------------------------------------------/

 Yosys 0.26+1 (git sha1 b1a011138, x86_64-w64-mingw32-g++ 9.2.1 -Os)


yosys> exit
End of script. Logfile hash: da39a3ee5e
Yosys 0.26+1 (git sha1 b1a011138, x86_64-w64-mingw32-g++ 9.2.1 -Os)
Time spent: no commands executed
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2>
```

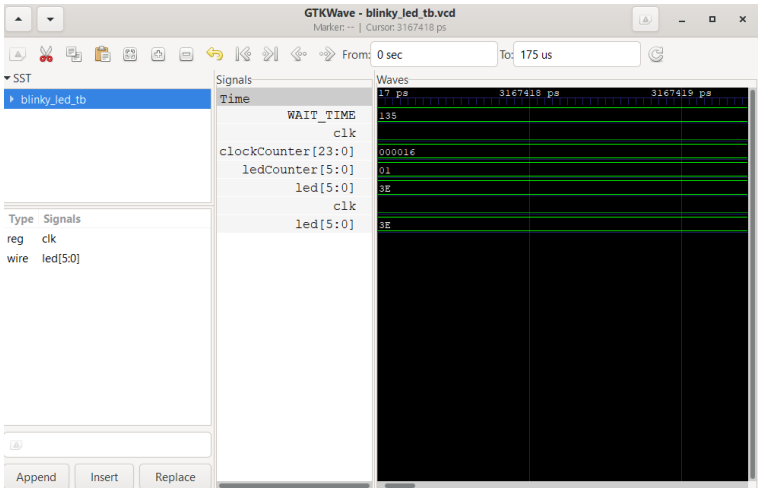
Uso del código “cd .\ejemplos\BlinkyLed\src\build\”:

```
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2\open_source_fpga_environment> cd .\ejemplos\BlinkyLed\src\build\
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2\open_source_fpga_environment\ejemplos\BlinkyLed\src\build>
```

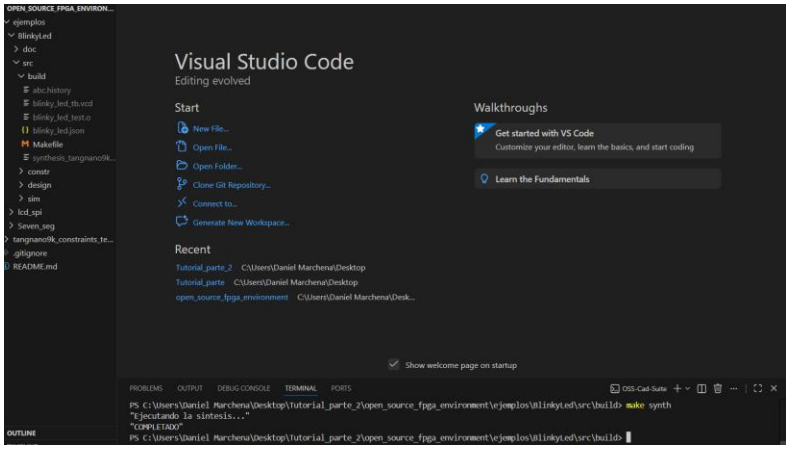
El testbench y simular el diseño (make test):

```
Time: 97910ns, LED: 011011
Time: 100630ns, LED: 011010
Time: 103350ns, LED: 011001
Time: 106070ns, LED: 011000
Time: 108790ns, LED: 010111
Time: 111510ns, LED: 010110
Time: 114230ns, LED: 010101
Time: 116950ns, LED: 010100
Time: 119670ns, LED: 010011
Time: 122390ns, LED: 010010
Time: 125110ns, LED: 010001
Time: 127830ns, LED: 010000
Time: 130550ns, LED: 001111
Time: 133270ns, LED: 001110
Time: 135990ns, LED: 001101
Time: 138710ns, LED: 001100
Time: 141430ns, LED: 001011
Time: 144150ns, LED: 001010
Time: 146870ns, LED: 001001
Time: 149590ns, LED: 001000
Time: 152310ns, LED: 000111
Time: 155030ns, LED: 000110
Time: 157750ns, LED: 000101
Time: 160470ns, LED: 000100
Time: 163190ns, LED: 000011
Time: 165910ns, LED: 000010
Time: 168630ns, LED: 000001
Time: 171350ns, LED: 000000
Time: 174070ns, LED: 111111
Ending simulation...
../sim/blinky_led_tb.sv:31: $finish called at 175000000 (1ps)
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2\open_source_fpga_environment\ejemplos\BlinkyLed\src\build>
```

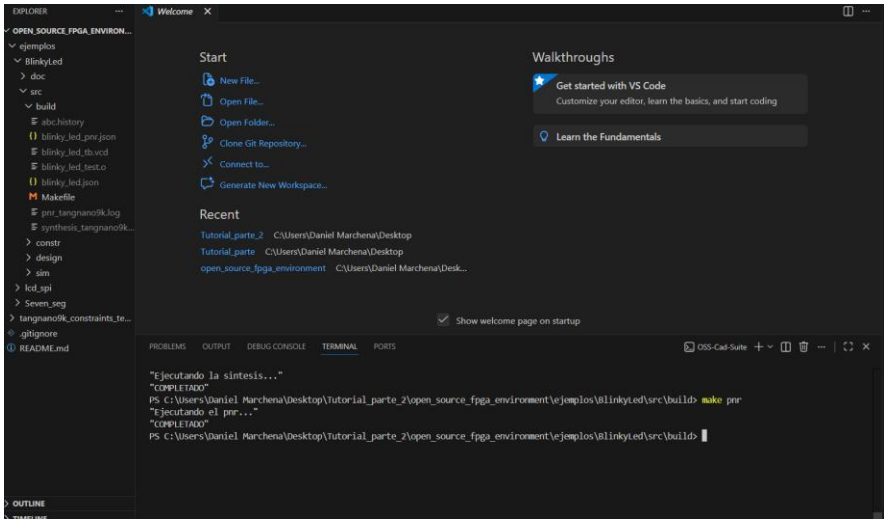
Visualizar los diagramas de tiempo con GTKwave:



Para verificar la sintaxis y sintetizar los diseños RTL (make synth):



Para realizar el place and route del diseño sintetizado en la FPGA:



Para generar el bitstream y load que contiene el diseño:

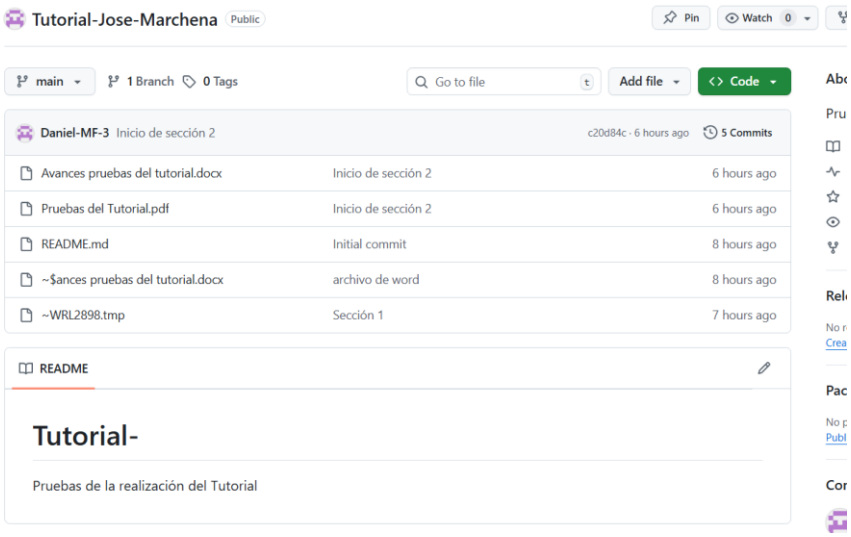
```
"Generando blinky_led_tangnano9k.fs"
<frozen importlib._bootstrap>:219: Warning: Numpy built with MINGW-W64 on Windows 64 bits is experimental, and only available for
testing. You are advised not to use it for production.

CRASHES ARE TO BE EXPECTED - PLEASE REPORT THEM TO NUMPY DEVELOPERS
"COMPLETADO"
openFPGALoader -b tangnano9k blinky_led_tangnano9k.fs
Jtag frequency : requested 6.00MHz   -> real 6.00MHz
Parse file Parse blinky_led_tangnano9k.fs:
Done
DONE
Jtag frequency : requested 2.50MHz   -> real 2.00MHz
erase SRAM Done
Flash SRAM: [==================================================] 100.00%
Done
SRAM Flash: Success
PS C:\Users\Daniel Marchena\Desktop\Tutorial_parte_2\open_source_fpga_environment\ejemplos\BlinkyLed\src\build>
```

Make all:

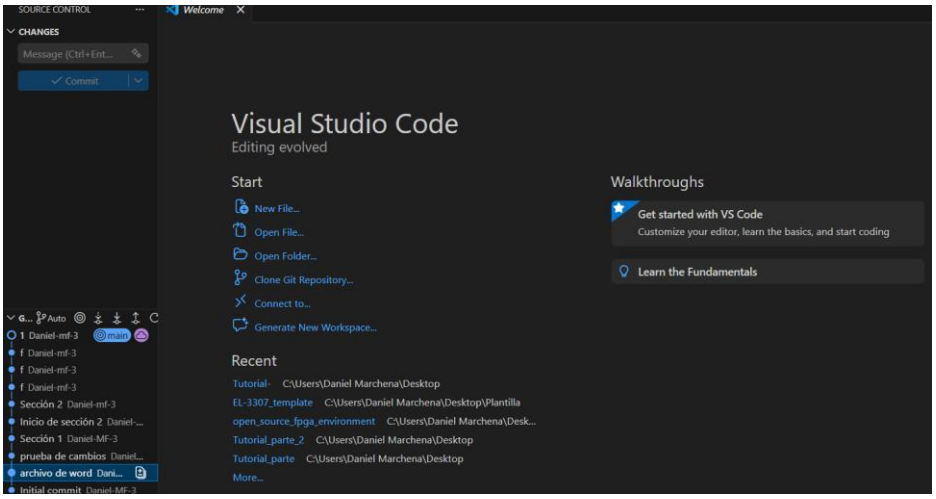
```
"Ejecutando la sintesis..."
"COMPLETADO"
"Ejecutando el pnr..."
"COMPLETADO"
"Generando blinky_led_tangnano9k.fs"
<frozen importlib._bootstrap>:219: Warning: Numpy built with MINGW-W64 on Windows 64 bits is experimental, and only available for
testing. You are advised not to use it for production.

CRASHES ARE TO BE EXPECTED - PLEASE REPORT THEM TO NUMPY DEVELOPERS
"COMPLETADO"
openFPGALoader -b tangnano9k blinky_led_tangnano9k.fs
Jtag frequency : requested 6.00MHz   -> real 6.00MHz
Parse file Parse blinky_led_tangnano9k.fs:
Done
DONE
Jtag frequency : requested 2.50MHz   -> real 2.00MHz
erase SRAM Done
Flash SRAM: [==================================================] 100.00%
Done
SRAM Flash: Success
```

Sección 3

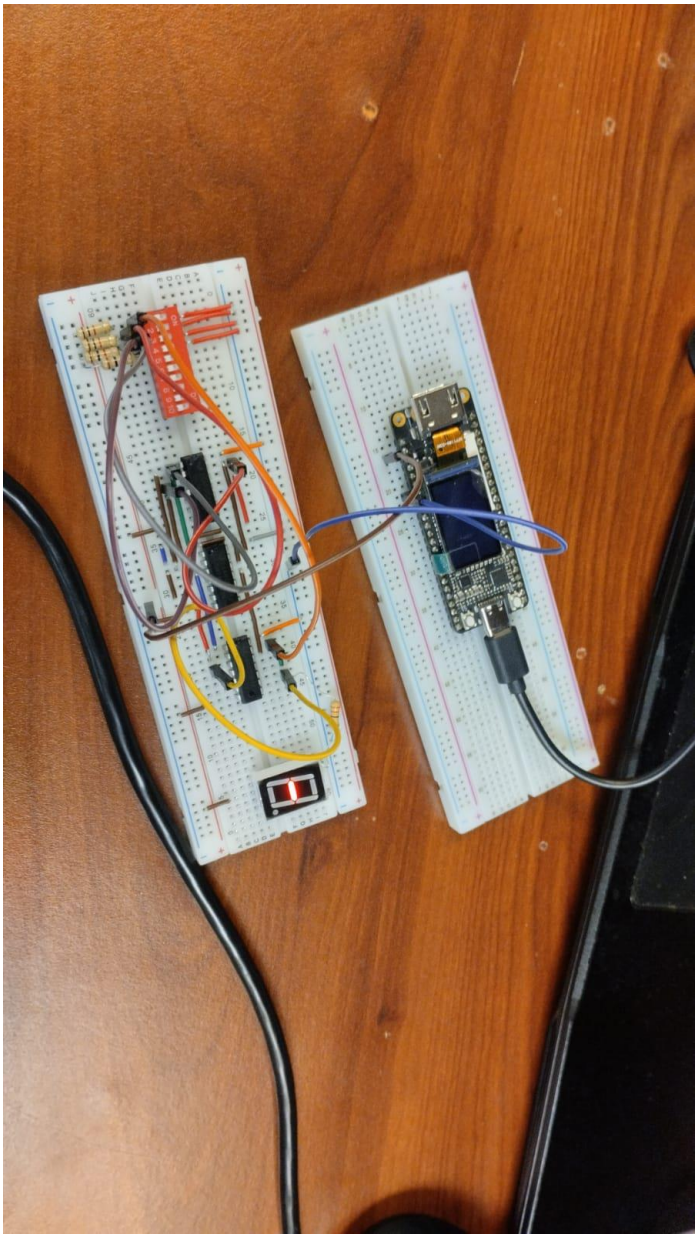
Creación de repositorio



Uso de commit:



Sección 4



Sección 5

Desing: creación de carpeta module_counter.sv:

module_counter.sv ● module_counter_tb.sv U ●

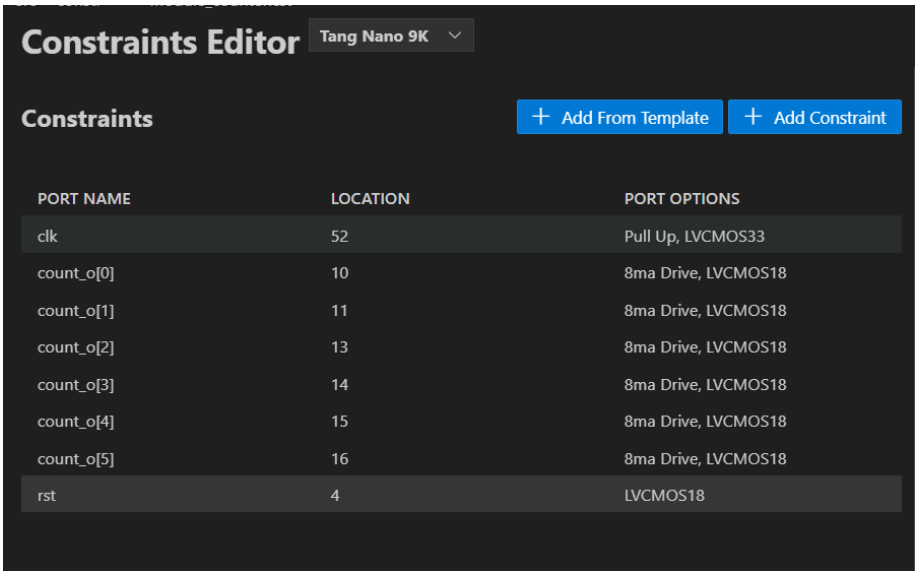
src > design > module_counter.sv

```
module module_counter # (

    parameter COUNT =13500000

)(
    input logic clk,
    input logic rst,

    output logic [5 : 0] count_o
);

 localparam WIDTH_COUT = $clog2(COUNT);

 logic [WIDTH_COUT - 1 : 0]clk_counter='0;
 logic [5 : 0] led_countr_r;

 always_ff @(posedge clk) begin

    if(!rst)begin
        led_countr_r <= '0;
        clk_counter <= '0;
    end

    else if (clk_counter == COUNT - 1)begin
        clk_counter <= '0;
        led_countr_r <= led_countr_r + 1'b1;
    end

    else begin
        clk_counter <=  clk_counter + 1'b1;
        led_countr_r  <= led_countr_r;
    end

 end

 assign count_o= ~led_countr_r;
```

Sim: Creación de carapeta module_counter_tb.sv:

```
`timescale 1ns/1ps

module module_counter_tb;

logic clk;
logic rst;
logic [5 : 0]count_o;

module_counter # (10) COUNTER (
    .clk (clk),
    .rst (rst),
    ,count_o(count_o)

);

initial begin
    clk = 0;
    rst = 0;

    #30;

    rst = 0;

    #30;
    rst = 1;

    # 300000;
    $finish
end

always begin
    clk=~clk
    #10;
end

initial begin
    $dumpfile("module_counter_tb.vcd");
```

Constraints



PORT NAME	LOCATION	PORT OPTIONS
clk	52	Pull Up, LVCMOS33
count_o[0]	10	8ma Drive, LVCMOS18
count_o[1]	11	8ma Drive, LVCMOS18
count_o[2]	13	8ma Drive, LVCMOS18
count_o[3]	14	8ma Drive, LVCMOS18
count_o[4]	15	8ma Drive, LVCMOS18
count_o[5]	16	8ma Drive, LVCMOS18
rst	4	LVCMOS18

Ejecutar: